

tf.compat.v1.profiler.ProfileOptionBuilder

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/profiler/ProfileOptionBuilder

Option Builder for Profiling API.

Function Signatures

```
tf.compat.v1.profiler.ProfileOptionBuilder( options=None )  
account_displayed_op_only( is_true )
```

Code Examples

```
tf.compat.v1.profiler.ProfileOptionBuilder(  
options=None  
)
```

```
tf.compat.v1.profiler.ProfileOptionBuilder(  
options=None  
)
```

```
# Users can use pre-built options:
opts = (
    tf.profiler.ProfileOptionBuilder.trainable_variables_parameter())
# Or, build your own options:
opts = (tf.compat.v1.profiler.ProfileOptionBuilder()
        .with_max_depth(10)
        .with_min_micros(1000)
        .select(['accelerator_micros']))
        .with_stdout_output()
        .build())
# Or customize the pre-built options:
opts = (tf.compat.v1.profiler.ProfileOptionBuilder(
    tf.profiler.ProfileOptionBuilder.time_and_memory())
        .with_displaying_options(show_name_regexes=['.*rnn.*'])
        .build())
# Finally, profiling with the options:
_ = tf.compat.v1.profiler.profile(tf.compat.v1.get_default_graph(),
    run_meta=run_meta,
    cmd='scope',
    options=opts)
```

```
# Users can use pre-built options:
opts = (
    tf.profiler.ProfileOptionBuilder.trainable_variables_parameter()
# Or, build your own options:
opts = (tf.compat.v1.profiler.ProfileOptionBuilder()
    .with_max_depth(10)
    .with_min_micros(1000)
    .select(['accelerator_micros'])
    .with_stdout_output()
    .build())
# Or customize the pre-built options:
opts = (tf.compat.v1.profiler.ProfileOptionBuilder(
    tf.profiler.ProfileOptionBuilder.time_and_memory())
    .with_displaying_options(show_name_regexes=['.*rnn.*'])
    .build())
# Finally, profiling with the options:
_ = tf.compat.v1.profiler.profile(tf.compat.v1.get_default_graph(),
    run_meta=run_meta,
    cmd='scope',
    options=opts)
```

```
account_displayed_op_only(
    is_true
)
```

```
account_displayed_op_only(
    is_true
)
```

Documentation

Option Builder for Profiling API.

For tutorial on the options, see

<https://github.com/tensorflow/tensorflow/tree/master/tensorflow/core/profiler/g3doc/op>

Methods

`## account_displayed_op_only`

[View source](#)

Whether only account the statistics of displayed profiler nodes.

[View source](#)

Build a profiling option.

float_operation

[View source](#)

Options used to profile float operations.

Please [see](https://github.com/tensorflow/tensorflow/tree/master/tensorflow/core/profiler/g3doc/profiling.md)
<https://github.com/tensorflow/tensorflow/tree/master/tensorflow/core/profiler/g3doc/profiling.md>
on the caveats of calculating float operations.

order_by

[View source](#)

Order the displayed profiler nodes based on a attribute.

Supported attribute includes micros, bytes, occurrence, params, etc.
<https://github.com/tensorflow/tensorflow/tree/master/tensorflow/core/profiler/g3doc/profiling.md>

select

[View source](#)

Select the attributes to display.

See

<https://github.com/tensorflow/tensorflow/tree/master/tensorflow/core/profiler/g3doc/op>
for supported attributes.

time_and_memory

[View source](#)

Show operation time and memory consumptions.

trainable_variables_parameter

[View source](#)

Options used to profile trainable variable parameters.

Normally used together with 'scope' view.

with_accounted_types

[View source](#)

Selectively counting statistics based on node types.

Here, 'types' means the profiler nodes' properties. Profiler by default consider device name (e.g. /job:xx/.../device:GPU:0) and operation type (e.g. MatMul) as profiler nodes' properties. User can also associate customized 'types' to profiler nodes through OpLogProto proto.

For example, user can select profiler nodes placed on gpu:0 with:

```
account_type_regexes=['.*gpu:0.*']
```

If none of a node's properties match the specified regexes, the node is not displayed nor accounted.

with_empty_output

[View source](#)

Do not generate side-effect outputs.

with_file_output

[View source](#)

Print the result to a file.

with_max_depth

[View source](#)

Set the maximum depth of display.

The depth depends on profiling view. For 'scope' view, it's the depth of name scope hierarchy (tree), for 'op' view, it's the number of operation types (list), etc.

with_min_execution_time

[View source](#)

Only show profiler nodes consuming no less than 'min_micros'.

with_min_float_operations

[View source](#)

Only show profiler nodes consuming no less than 'min_float_ops'.

Please [see](https://github.com/tensorflow/tensorflow/tree/master/tensorflow/core/profiler/g3doc/profiler.md) <https://github.com/tensorflow/tensorflow/tree/master/tensorflow/core/profiler/g3doc/profiler.md> on the caveats of calculating float operations.

with_min_memory

[View source](#)

Only show profiler nodes consuming no less than 'min_bytes'.

with_min_occurrence

[View source](#)

Only show profiler nodes including no less than 'min_occurrence' graph nodes.

A "node" means a profiler output node, which can be a python line (code view), an operation type (op view), or a graph node (graph/scope view). A python line includes all graph nodes created by that line, while an operation type includes all graph nodes of that type.

with_min_parameters

[View source](#)

Only show profiler nodes holding no less than 'min_params' parameters.

'Parameters' normally refers the weights of in TensorFlow variables. It reflects the 'capacity' of models.

with_node_names

[View source](#)

Regular expressions used to select profiler nodes to display.

After 'with_accounted_types' is evaluated, 'with_node_names' are evaluated as follows:

For a profile data structure, profiler first finds the profiler nodes matching 'start_name_regexes', and starts displaying profiler nodes from there. Then, if a node matches 'show_name_regexes' and doesn't match 'hide_name_regexes', it's displayed. If a node matches 'trim_name_regexes', profiler stops further searching that branch.

with_pprof_output

[View source](#)

Generate a pprof profile gzip file.

with_stdout_output

[View source](#)

Print the result to stdout.

with_step

[View source](#)

Which profile step to use for profiling.

The 'step' here refers to the step defined by `Profiler.add_step()` API.

with_timeline_output

[View source](#)

Generate a timeline json file.